# Modello FSP e Reti di Petri di un ascensore con utenti e allarme

**Assignment – 01**

## 1. Descrizione generale del modello

Il sistema modellato è un ascensore di un edificio con N piani, utilizzato da più utenti che lo chiamano ai piani, scelgono la destinazione in cabina e vengono serviti in ordine di arrivo (FIFO). Un segnale di allarme antincendio interrompe l'attività corrente: l'ascensore scende al piano terra, apre la porta e resta fermo finché non arriva il fine allarme.

## 2. Scelte di modellazione

- Istanza del modello: N piani (N = 3) e 2 utenti (a, b). Più utenti possono chiamare lo stesso piano (a e b possono scegliere lo stesso valore di f), quindi la presenza di più utenti per piano è rappresentata. La coda FIFO è rispettata per i due utenti.
- Movimento astratto: l'azione *arrive[f]* indica «l'ascensore è arrivato al piano f» senza tracciare la posizione corrente della cabina. L'identità del piano non influisce sul flusso di controllo: i piani sono solo etichette delle azioni.
- Capacità 1: l'ascensore serve un solo utente per volta, perché torna in `IDLE` solo dopo l'uscita (o l'evacuazione) dell'utente.
- FIFO: la coda registra l'ordine delle chiamate e l'ascensore può servire solo l'utente in testa.
- Allarme: l'ascensore interrompe l'attività corrente, va al piano terra (*go_ground*), apre la porta (*open_door*), annulla la chiamata o evacua l'utente coinvolto e resta fermo finché non arriva `solved`.

## 3. Modello FSP

### 3.1 Costanti e utente

Il piano 0 è il piano terra. Ogni utente percorre il ciclo di vita seguente (le azioni vengono prefissate con `a.` o `b.` dalla composizione `USERS`):

- *call[f]*: preme il pulsante di chiamata al piano f;
- *arrive[f]*: l'ascensore è arrivato al piano f;
- *enter*: l'utente entra in cabina;
- *select[d]*: preme il pulsante del piano di destinazione d;
- *exit[d]*: esce al piano d;
- *cancel*: (solo per allarme) la chiamata o l'attesa viene annullata;
- *evacuate*: (solo per allarme) l'utente viene fatto uscire.

```
const N = 3
range Floors = 0..N-1

USER = (call[f:Floors] -> WAITING[f]),
WAITING[f:Floors] = (arrive[f] -> AT_DOOR | cancel -> USER),
AT_DOOR = (enter -> INSIDE | cancel -> USER),
INSIDE  = (select[d:Floors] -> RIDING[d] | evacuate -> USER),
RIDING[d:Floors] = (exit[d] -> USER | evacuate -> USER).

||USERS = (a:USER || b:USER).
```

### 3.2 Coda FIFO delle chiamate

La coda registra l'ordine delle chiamate: l'ascensore può servire (*serve*) solo l'utente che ha chiamato per primo. Gli stati sono: `EMPTY` (nessuno in coda), `QA` (solo a), `QB` (solo b), `QAB` (a in testa, b dietro) e `QBA` (b in

testa, a dietro). Un utente già in coda non può chiamare di nuovo (è comunque bloccato in `WAITING`), quindi non servono altre transizioni.

```
QUEUE = EMPTY,
EMPTY = (a.call[Floors] -> QA | b.call[Floors] -> QB),
QA    = (a.serve -> EMPTY | b.call[Floors] -> QAB),
QB    = (b.serve -> EMPTY | a.call[Floors] -> QBA),
QAB   = (a.serve -> QB),     // a ha chiamato prima di b
QBA   = (b.serve -> QA).     // b ha chiamato prima di a
```

### 3.3 Allarme antincendio

L'allarme scatta (`alarm`) e rimane attivo finché un segnale esterno ne comunica la risoluzione (`solved`); poi può scattare di nuovo.

```
ALARM = (alarm -> solved -> ALARM).
```

### 3.4 Ascensore

L'ascensore serve un utente alla volta: da `IDLE` prende il primo della coda (*serve*) e torna in `IDLE` solo quando l'utente esce. In qualsiasi stato può ricevere `alarm`:

- se l'utente non è ancora entrato (`A_GO`, `A_DOOR`), la sua chiamata viene annullata (*cancel*) e l'ascensore va al piano terra;
- se l'utente è in cabina (`A_IN`, `A_RIDE`), l'ascensore va al piano terra, apre la porta ed evacua l'utente;
- se l'ascensore è libero (`IDLE`), va al piano terra e apre la porta; gli utenti in coda mantengono la loro posizione.

In ogni caso l'ascensore attende `solved` nello stato `STOPPED` e poi torna operativo. Di seguito è riportata la parte relativa all'utente a; quella per l'utente b è perfettamente simmetrica (`B_GO`, `B_DOOR`, `B_IN`, `B_RIDE`).

```
LIFT = IDLE,
IDLE = (a.serve -> A_GO | b.serve -> B_GO | alarm -> GROUND),

A_GO   = (a.arrive[f:Floors] -> A_DOOR | alarm -> a.cancel -> GROUND),
A_DOOR = (a.enter -> A_IN | alarm -> a.cancel -> GROUND),
A_IN   = (a.select[d:Floors] -> A_RIDE[d]
         | alarm -> go_ground -> open_door -> a.evacuate -> STOPPED),
A_RIDE[d:Floors] = (a.exit[d] -> IDLE
         | alarm -> go_ground -> open_door -> a.evacuate -> STOPPED),
// ... B_GO, B_DOOR, B_IN, B_RIDE analoghi con b

GROUND  = (go_ground -> open_door -> STOPPED),
STOPPED = (solved -> IDLE).
```

### 3.5 Sistema completo

```
||LIFT_SYSTEM = (USERS || QUEUE || LIFT || ALARM).
```

La composizione parallela sincronizza i processi sulle azioni in comune: ad esempio *a.call[f]* è condivisa da `USER` e `QUEUE`, *a.serve* da `QUEUE` e `LIFT`, *a.arrive[f]* da `USER` e `LIFT`, *alarm* e *solved* da `LIFT` e `ALARM`.

## 4. Modello a Reti di Petri (punto 2)

### 4.1 Struttura della rete

La rete ricalca la struttura del modello FSP. Ogni processo sequenziale diventa una componente a macchina a stati: un solo gettone percorre i posti di quel componente, e i vincoli di sincronizzazione dell'FSP diventano transizioni con più posti in ingresso. La rete è una rete Posti/Transizioni ordinaria (non colorata). Per mantenerla di dimensioni contenute l'identità del piano viene astratta: nel modello FSP i piani sono semplici etichette che non influenzano il flusso di controllo, quindi *call[f]*, *arrive[f]*, *select[d]* ed *exit[d]* sono rappresentate da una sola transizione ciascuna.

La rete ha 28 posti e 27 transizioni.

| Componente | Posti | Significato |
|---|---|---|
| Utente u ∈ {a, b} | u_idle, u_wait, u_door, u_in, u_ride | fuori dal sistema; ha chiamato e aspetta; porta aperta; in cabina; in viaggio verso la destinazione |
| Coda FIFO | Q_E, Q_A, Q_B, Q_AB, Q_BA | un solo gettone: stato della coda, come in QUEUE |
| Ascensore | L_idle, L_goA, L_doorA, L_inA, L_rideA, L_goB, L_doorB, L_inB, L_rideB, L_ground, L_stopped | un solo gettone: stato dell'ascensore, come in LIFT |
| Allarme | Al_off, Al_on | allarme non attivo / attivo |

**Marcatura iniziale:** un gettone in a_idle, b_idle, Q_E, L_idle e Al_off.

### 4.2 Transizioni

La tabella riporta, per ogni transizione, i posti in ingresso e in uscita. Con u ∈ {a, b} si indica l'utente e con U ∈{A, B} il corrispondente indice nei posti dell'ascensore.

| Transizione | Ingresso | Uscita | Azione FSP |
|---|---|---|---|
| call_a_1 | a_idle, Q_E | a_wait, Q_A | *a.call* (coda vuota) |
| call_a_2 | a_idle, Q_B | a_wait, Q_BA | *a.call* (b già in coda) |
| call_b_1 | b_idle, Q_E | b_wait, Q_B | *b.call* (coda vuota) |
| call_b_2 | b_idle, Q_A | b_wait, Q_AB | *b.call* (a già in coda) |
| serve_a_1 | L_idle, Q_A | L_goA, Q_E | *a.serve* (a sola in coda) |
| serve_a_2 | L_idle, Q_AB | L_goA, Q_B | *a.serve* (a in testa) |
| serve_b_1 | L_idle, Q_B | L_goB, Q_E | *b.serve* (b solo in coda) |
| serve_b_2 | L_idle, Q_BA | L_goB, Q_A | *b.serve* (b in testa) |
| arrive_u | L_goU, u_wait | L_doorU, u_door | *u.arrive[f]* |
| enter_u | L_doorU, u_door | L_inU, u_in | *u.enter* |
| select_u | L_inU, u_in | L_rideU, u_ride | *u.select[d]* |
| exit_u | L_rideU, u_ride | L_idle, u_idle | *u.exit[d]* |
| alarm_idle | L_idle, Al_off | L_ground, Al_on | *alarm* da IDLE |
| alarm_go_u | L_goU, u_wait, Al_off | L_ground, u_idle, Al_on | *alarm* + *u.cancel* |
| alarm_door_u | L_doorU, u_door, Al_off | L_ground, u_idle, Al_on | *alarm* + *u.cancel* |
| alarm_in_u | L_inU, u_in, Al_off | L_ground, u_idle, Al_on | *alarm* + *u.evacuate* |
| alarm_ride_u | L_rideU, u_ride, Al_off | L_ground, u_idle, Al_on | *alarm* + *u.evacuate* |
| go_open | L_ground | L_stopped | *go_ground*, *open_door* |
| solved | L_stopped, Al_on | L_idle, Al_off | *solved* |

Rispetto all'FSP la rete compie due semplificazioni, che non ne alterano il comportamento rilevante: le azioni *go_ground* e *open_door* sono fuse in un'unica transizione go_open, e l'evento di allarme comprende atomicamente l'annullamento della chiamata o l'evacuazione dell'utente (le transizioni alarm_*_u).

### 4.3 Rappresentazione grafica

Per leggibilità la rete è mostrata in tre parti; i posti tratteggiati sono condivisi con le altre figure. La Figura 1 riporta il ciclo di vita dell'utente a insieme alla corrispondente catena dell'ascensore (l'utente b è identico, con

posti `b_*`, `L_goB`, …). La Figura 2 riporta la coda FIFO, che riceve i gettoni dalle transizioni `call_*` e `serve_*`. La Figura 3 riporta la gestione dell'allarme; ognuna delle 9 transizioni `alarm_*` consuma `Al_off` e un posto dell'ascensore con il relativo posto dell'utente, come dettagliato nella tabella.

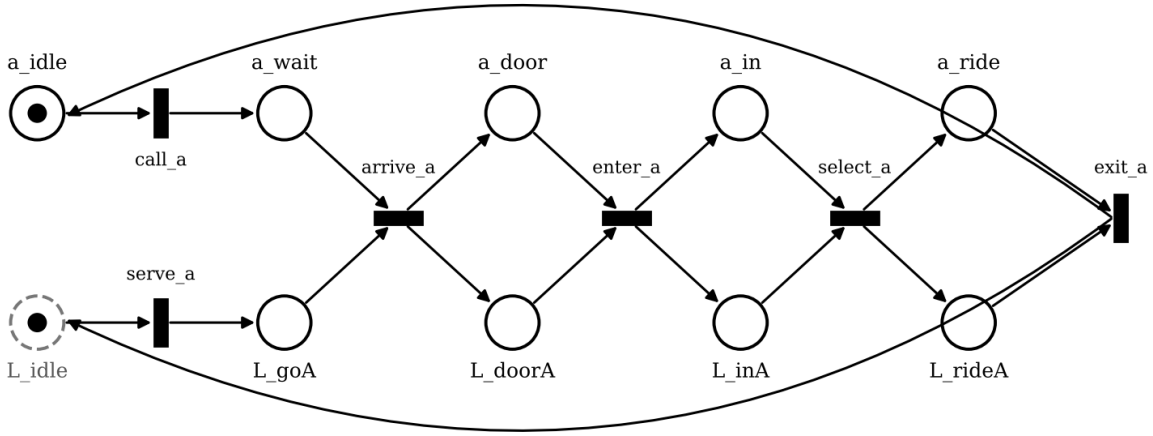



*Figura 1 – Ciclo dell'utente a e catena dell'ascensore corrispondente.*

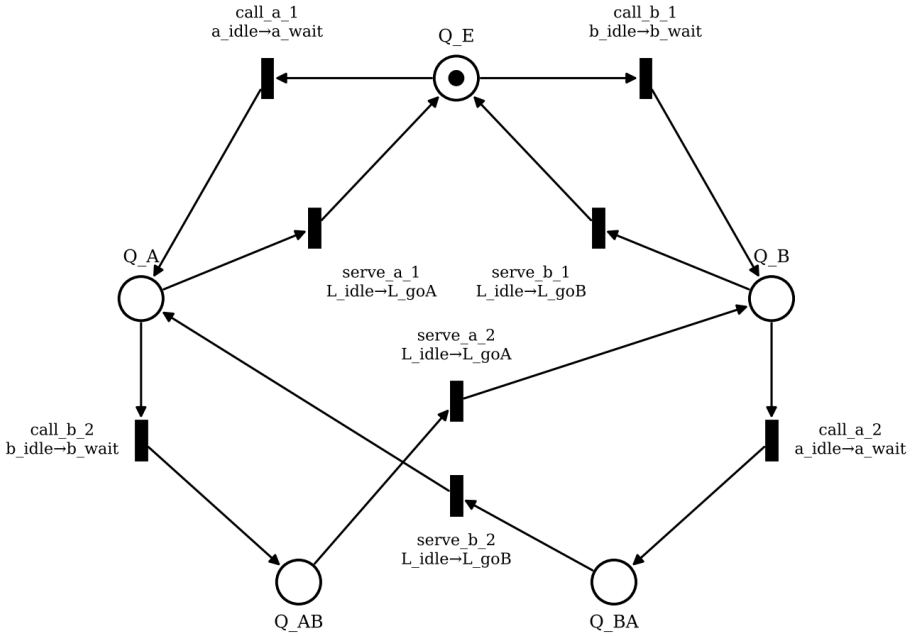



*Figura 2 – Coda FIFO (le transizioni `call_*` e `serve_*` agiscono anche su utenti e ascensore).*

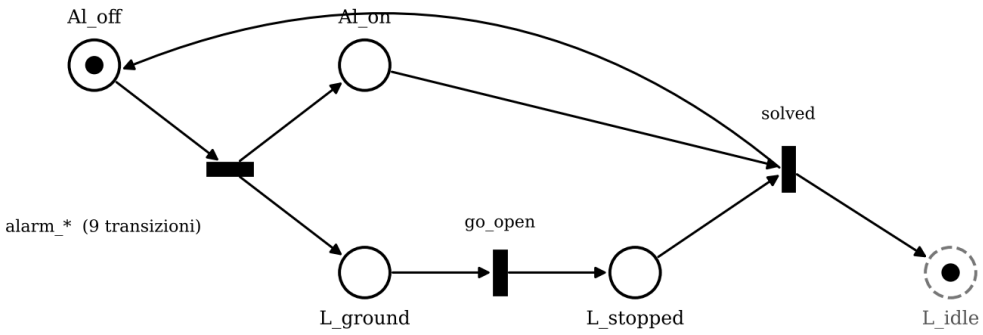



*Figura 3 – Gestione dell'allarme (nodi condivisi con le Figure 1 e 2).*

### 4.4 Analisi della rete

Il grafo di raggiungibilità della rete è stato calcolato per enumerazione a partire dalla marcatura iniziale. Risulta che:

- le marcature raggiungibili sono 31;
- la rete è 1-limitata (safe): nessun posto contiene mai più di un gettone;
- non esistono marcature morte (nessun deadlock) e tutte le 27 transizioni scattano almeno una volta;
- i posti di ciascun componente (coda, ascensore, allarme) contengono sempre esattamente un gettone, quindi coda, ascensore e allarme si trovano sempre in un unico stato;
- **mutua esclusione:** i posti `a_in`, `a_ride` e `b_in`, `b_ride` non sono mai marcati contemporaneamente. Infatti l'ascensore ha un solo gettone e a ogni posto utente in cabina corrisponde il posto ascensore equivalente (ad esempio `a_in` è marcato se e solo se `L_inA` è marcato).

## 5. Proprietà FLTL e verifica con LTSA (punto 3)

### 5.1 Fluenti

Un fluente è una proposizione che diventa vera quando accade un'azione «iniziale» e torna falsa quando accade una «terminale»; serve per esprimere le proprietà FLTL sullo stato del sistema. I fluenti sono tutti falsi all'inizio.

| Fluente | Iniziato da | Terminato da |
|---|---|---|
| `ALARM_ON` | *alarm* | *solved* |
| `A_IN_LIFT` / `B_IN_LIFT` | *u.enter* | *u.exit[Floors]*, *u.evacuate* |
| `A_CALLED` / `B_CALLED` | *u.call[Floors]* | *u.arrive[Floors]*, *u.cancel* |
| `A_SELECTED` / `B_SELECTED` | *u.select[Floors]* | *u.exit[Floors]*, *u.evacuate* |
| `A_WAITING` / `B_WAITING` | *u.call[Floors]* | *u.enter*, *u.cancel* |

```
fluent ALARM_ON   = <alarm, solved>
fluent A_IN_LIFT  = <a.enter, {a.exit[Floors], a.evacuate}>
fluent A_CALLED   = <a.call[Floors], {a.arrive[Floors], a.cancel}>
fluent A_SELECTED = <a.select[Floors], {a.exit[Floors], a.evacuate}>
fluent A_WAITING  = <a.call[Floors], {a.enter, a.cancel}>
// ... e gli analoghi per b
```

### 5.2 Proprietà

**1) Un solo utente alla volta usa l'ascensore.** È espressa sia come asserzione FLTL sia come *property* di safety, verificabile con il composito `CHECK_ONE_AT_A_TIME`.

```
assert MUTEX = []!(A_IN_LIFT && B_IN_LIFT)

property ONE_AT_A_TIME = (a.enter -> A_INSIDE | b.enter -> B_INSIDE),
A_INSIDE = (a.exit[Floors] -> ONE_AT_A_TIME | a.evacuate -> ONE_AT_A_TIME),
B_INSIDE = (b.exit[Floors] -> ONE_AT_A_TIME | b.evacuate -> ONE_AT_A_TIME).

||CHECK_ONE_AT_A_TIME = (LIFT_SYSTEM || ONE_AT_A_TIME).
```

**2) Se un utente chiama l'ascensore, prima o poi questo arriva (tranne in caso di allarme).** Ogni volta che la chiamata è pendente, prima o poi o la chiamata cessa di essere pendente (l'ascensore è arrivato) oppure c'è un allarme.

```
assert A_LIFT_ARRIVES = [](A_CALLED -> <>(!A_CALLED || ALARM_ON))
assert B_LIFT_ARRIVES = [](B_CALLED -> <>(!B_CALLED || ALARM_ON))
```

**3) Se un utente in cabina seleziona un piano, prima o poi ci arriva (tranne in caso di allarme).**

```
assert A_SELECT_ARRIVES = [](A_SELECTED -> <>(!A_SELECTED || ALARM_ON))
assert B_SELECT_ARRIVES = [](B_SELECTED -> <>(!B_SELECTED || ALARM_ON))
```

**4) Un utente in attesa al piano prima o poi ottiene l'ascensore (tranne in caso di allarme).** `A_WAITING` termina con *enter* (l'utente ha ottenuto l'ascensore) oppure con *cancel*, che può avvenire solo per un allarme. Per uniformità con i punti 2 e 3, la formula è scritta usando solo fluenti. La politica FIFO della coda garantisce che nessun utente venga scavalcato all'infinito.

```
assert A_GETS_LIFT = [](A_WAITING -> <>(!A_WAITING || ALARM_ON))
assert B_GETS_LIFT = [](B_WAITING -> <>(!B_WAITING || ALARM_ON))
```

Nelle proprietà 2–4 la clausola «tranne in caso di allarme» è resa con il disgiunto `ALARM_ON`: la proprietà è soddisfatta se, dopo la richiesta, si verifica comunque un allarme. Questo corrisponde alla lettura richiesta dalla consegna (in caso di allarme l'ascensore interrompe l'attività e le richieste pendenti sono annullate o evacuate).

### 5.3 Procedura di verifica in LTSA

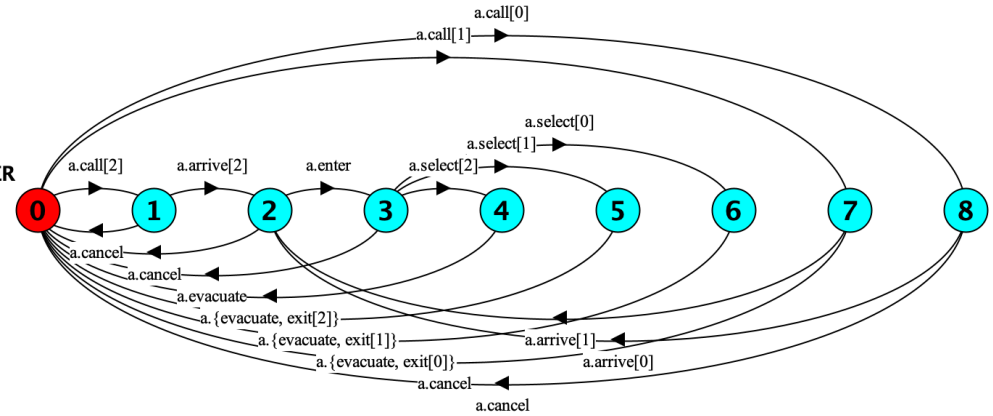
- Compilare il file (*Build* → *Compile*) e selezionare `LIFT_SYSTEM` come processo target.
- *Check* → *Safety* su `LIFT_SYSTEM`: ricerca di deadlock e di errori di safety.
- *Check* → *Safety* su `CHECK_ONE_AT_A_TIME`: verifica della *property* ONE_AT_A_TIME.
- *Check* → *LTL property* su `LIFT_SYSTEM`, selezionando a turno ciascuna asserzione: `MUTEX`, `A_LIFT_ARRIVES`, `B_LIFT_ARRIVES`, `A_SELECT_ARRIVES`, `B_SELECT_ARRIVES`, `A_GETS_LIFT`, `B_GETS_LIFT`.

### 5.4 Risultati

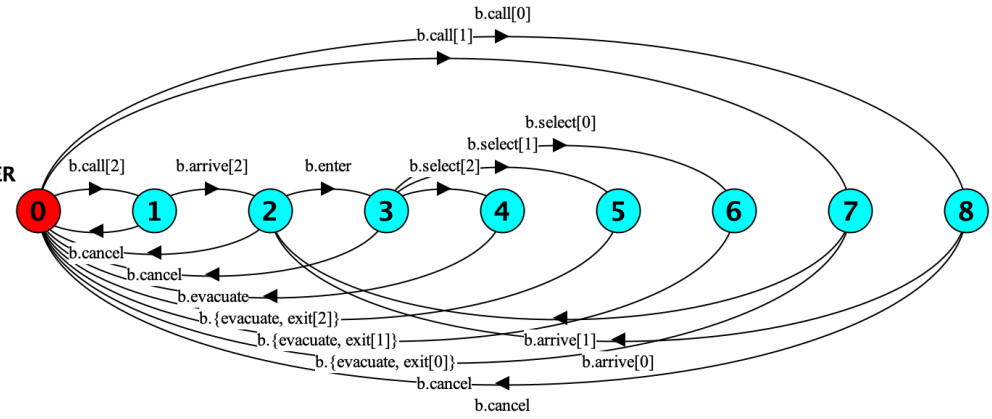
Il modello non presenta deadlock e tutte le proprietà risultano soddisfatte.

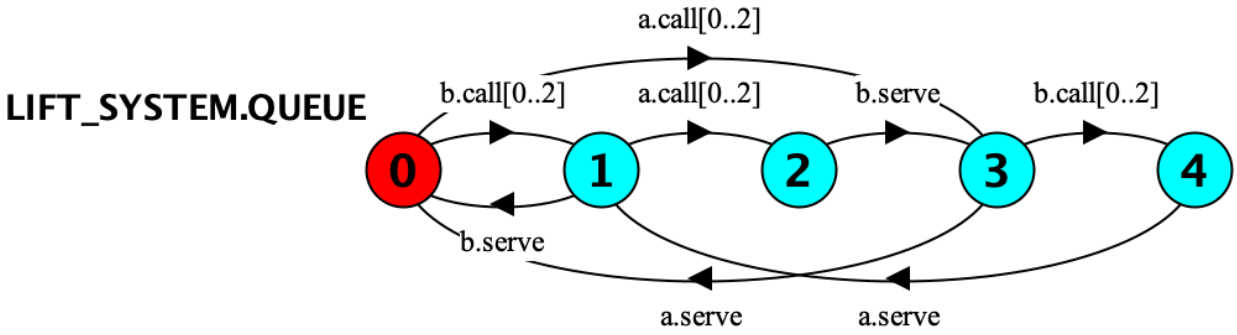
| Verifica | Comando LTSA | Esito |
|---|---|---|
| Assenza di deadlock | Safety su LIFT_SYSTEM | nessun deadlock |
| ONE_AT_A_TIME (property) | Safety su CHECK_ONE_AT_A_TIME | nessuna violazione |
| MUTEX | LTL property | soddisfatta |
| A_LIFT_ARRIVES, B_LIFT_ARRIVES | LTL property | soddisfatte |
| A_SELECT_ARRIVES, B_SELECT_ARRIVES | LTL property | soddisfatte |
| A_GETS_LIFT, B_GETS_LIFT | LTL property | soddisfatte |

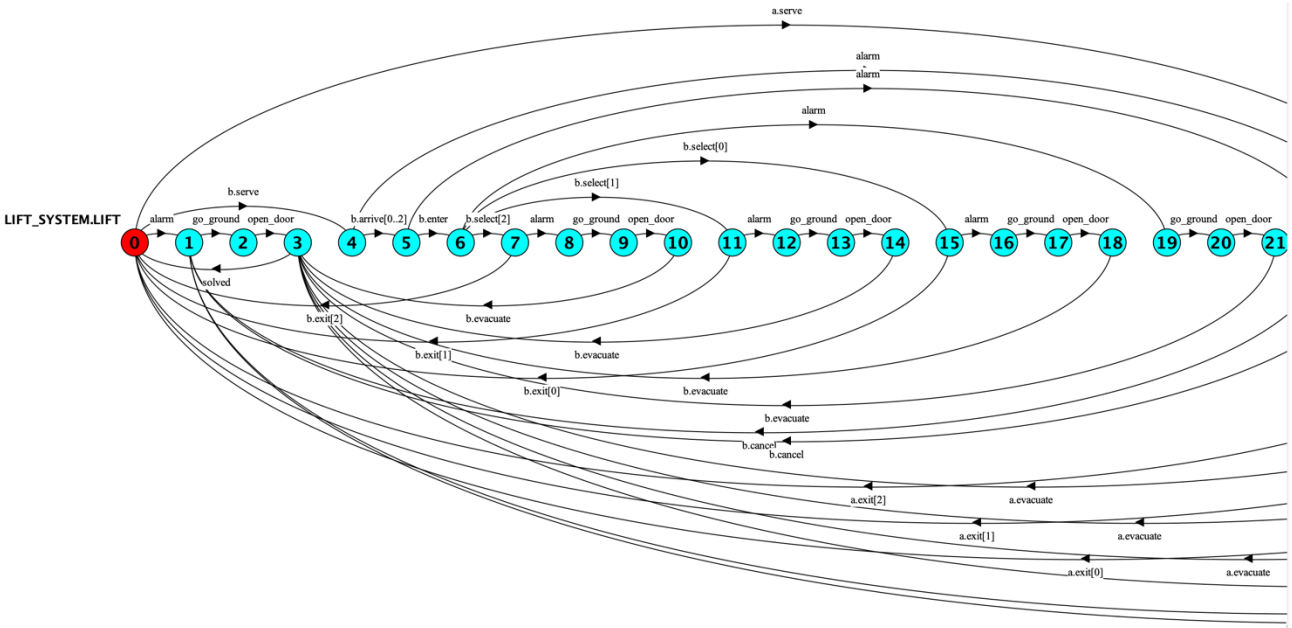
LIFT_SYSTEM.USERS.a:USER

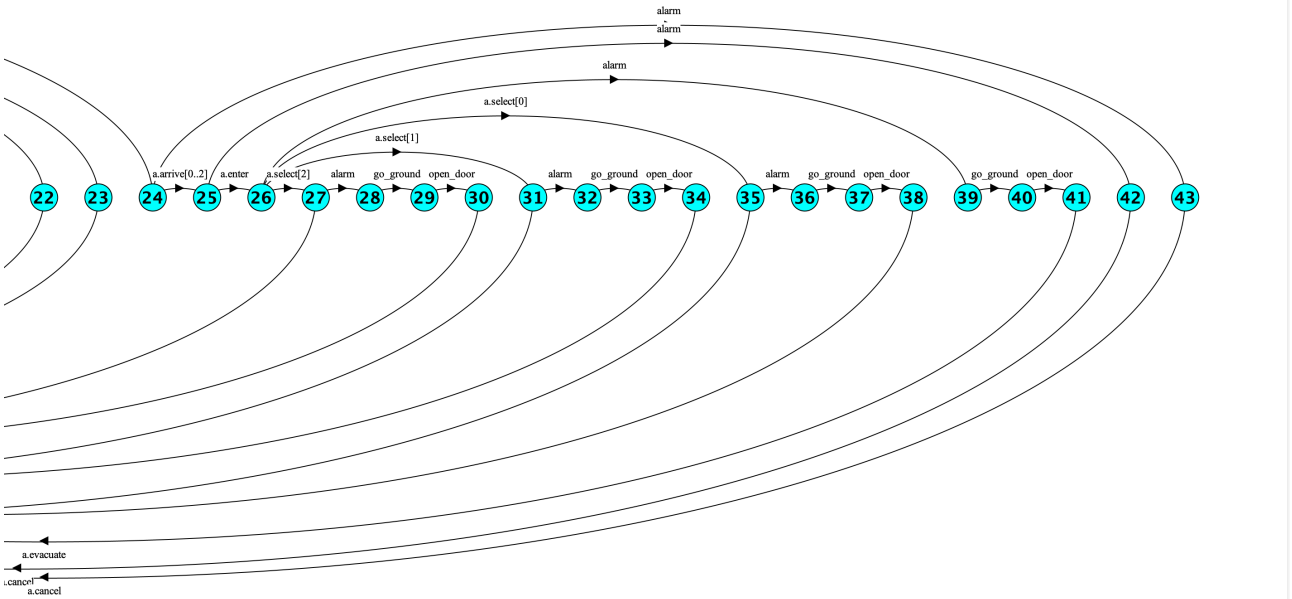


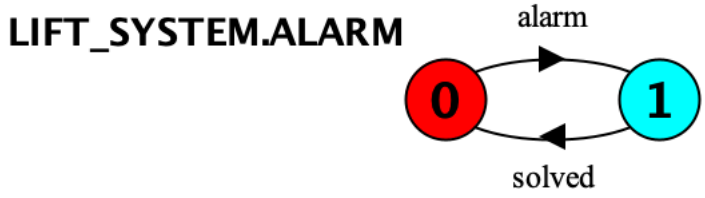

LIFT_SYSTEM.USERS.b:USER

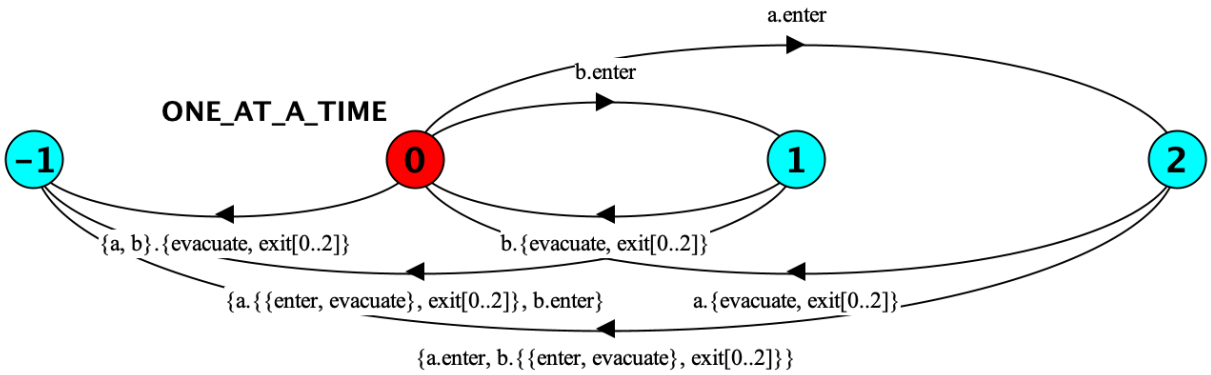



LIFT_SYSTEM.QUEUE
a.call[0..2]
b.call[0..2]
a.call[0..2]
b.serve
b.call[0..2]
0
1
2
3
4
b.serve
a.serve
a.serve


LIFT_SYSTEM.LIFT
a.serve
alarm
alarm
alarm
b.select[0]
b.select[1]
b.serve
alarm
go_ground
open_door
b.arrive[0..2]
b.enter
b.select[2]
solved
b.exit[2]
b.evacuate
b.exit[1]
b.evacuate
b.exit[0]
b.evacuate
b.evacuate
b.cancel
b.cancel
a.exit[2]
a.evacuate
a.exit[1]
a.evacuate
a.exit[0]
a.evacuate


alarm
alarm
alarm
a.select[0]
a.select[1]
a.arrive[0..2]
a.enter
a.select[2]
alarm
go_ground
open_door
a.evacuate
a.cancel
a.cancel




## 5.5 Output di LTSA

```
Compiled: USER
Compiled: QUEUE
Compiled: LIFT
Compiled: ALARM
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM
State Space:
 9 * 9 * 5 * 44 * 2 = 2 ** 18
Analysing...
Depth 10 -- States: 292 Transitions: 636 Memory used: 10330K
No deadlocks/errors
Analysed in: 2ms
```

---

*controllo safety su LIFT_SYSTEM: 292 stati, 636 transizioni e no deadlocks/errors*

```
Compiled: USER
Compiled: QUEUE
Compiled: LIFT
Compiled: ALARM
Compiled: ONE_AT_A_TIME
Composition:
CHECK_ONE_AT_A_TIME = LIFT_SYSTEM.USERS.a:USER || LIFT_SYSTEM.USERS.b:USER || LIFT_SYSTEM.QUEUE || LIFT_SYSTEM.LIFT || LIFT_SYSTEM.ALARM || ONE_AT_A_TIME
State Space:
 9 * 9 * 5 * 44 * 2 * 3 = 2 ** 20
Analysing...
Depth 10 -- States: 292 Transitions: 636 Memory used: 10367K
No deadlocks/errors
Analysed in: 1ms
```

*controllo safety su CHECK_ONE_AT_A_TIME: 292 stati, 636 transizioni e no deadlocks/errors*

```
Compiled: USER
Compiled: QUEUE
Compiled: LIFT
Compiled: ALARM
Formula !B_GETS_LIFT = (true U (B_WAITING & (false R (B_WAITING & !ALARM_ON))))
GBA 2 states 3 transitions
Buchi automata:
B_GETS_LIFT = S0,
S1@ =(B_WAITING&!ALARM_ON -> S1),
S0 =(true -> S0 |B_WAITING&!ALARM_ON -> S1).
Composition:
B_GETS_LIFT = B_WAITING || ALARM_ON || SYNC || B_GETS_LIFT
State Space:
 2 * 2 * 2 * 3 = 2 ** 5
Composing...
-- States: 20 Transitions: 88 Memory used: 10510K
Composed in 18ms
After Tau elimination = 9 state
B_GETS_LIFT minimising....
Minimised States: 6 in 1ms
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM || B_GETS_LIFT
State Space:
 9 * 9 * 5 * 44 * 2 * 6 = 2 ** 21
LTL Property Check...
-- States: 645 Transitions: 1688 Memory used: 12607K
No LTL Property violations detected.
LTL Property Check in: 3ms
```

```
Formula !B_LIFT_ARRIVES = (true U (B_CALLED & (false R (B_CALLED & !ALARM_ON))))
GBA 2 states 3 transitions
Buchi automata:
B_LIFT_ARRIVES = S0,
S1@ =(B_CALLED&!ALARM_ON -> S1),
S0 =(true -> S0 |B_CALLED&!ALARM_ON -> S1).
Composition:
B_LIFT_ARRIVES = B_CALLED || ALARM_ON || SYNC || B_LIFT_ARRIVES
State Space:
 2 * 2 * 2 * 3 = 2 ** 5
Composing...
-- States: 20 Transitions: 106 Memory used: 10179K
Composed in 20ms
After Tau elimination = 9 state
B_LIFT_ARRIVES minimising....
Minimised States: 6 in 0ms
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM || B_LIFT_ARRIVES
State Space:
 9 * 9 * 5 * 44 * 2 * 6 = 2 ** 21
LTL Property Check...
-- States: 641 Transitions: 1662 Memory used: 11228K
No LTL Property violations detected.
LTL Property Check in: 3ms
```

```
Formula !B_SELECT_ARRIVES = (true U (B_SELECTED & (false R (B_SELECTED & !ALARM_ON))))
GBA 2 states 3 transitions
Buchi automata:
B_SELECT_ARRIVES = S0,
S1@ =(B_SELECTED&!ALARM_ON -> S1),
S0 =(true -> S0 |B_SELECTED&!ALARM_ON -> S1).
Composition:
B_SELECT_ARRIVES = B_SELECTED || ALARM_ON || SYNC || B_SELECT_ARRIVES
State Space:
 2 * 2 * 2 * 3 = 2 ** 5
Composing...
-- States: 20 Transitions: 106 Memory used: 8087K
Composed in 19ms
After Tau elimination = 9 state
B_SELECT_ARRIVES minimising....
Minimised States: 6 in 0ms
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM || B_SELECT_ARRIVES
State Space:
 9 * 9 * 5 * 44 * 2 * 6 = 2 ** 21
LTL Property Check...
-- States: 596 Transitions: 1362 Memory used: 11232K
No LTL Property violations detected.
LTL Property Check in: 2ms
```

```
Formula !A_SELECT_ARRIVES = (true U (A_SELECTED & (false R (A_SELECTED & !ALARM_ON))))
GBA 2 states 3 transitions
Buchi automata:
A_SELECT_ARRIVES = S0,
S1@ =(A_SELECTED&!ALARM_ON -> S1),
S0 =(true -> S0 |A_SELECTED&!ALARM_ON -> S1).
Composition:
A_SELECT_ARRIVES = A_SELECTED || ALARM_ON || SYNC || A_SELECT_ARRIVES
State Space:
 2 * 2 * 2 * 3 = 2 ** 5
Composing...
-- States: 20 Transitions: 106 Memory used: 10205K
Composed in 16ms
After Tau elimination = 9 state
A_SELECT_ARRIVES minimising....
Minimised States: 6 in 0ms
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM || A_SELECT_ARRIVES
State Space:
 9 * 9 * 5 * 44 * 2 * 6 = 2 ** 21
LTL Property Check...
-- States: 596 Transitions: 1362 Memory used: 11254K
No LTL Property violations detected.
LTL Property Check in: 2ms
```

```
Formula !A_LIFT_ARRIVES = (true U (A_CALLED & (false R (A_CALLED & !ALARM_ON))))
GBA 2 states 3 transitions
Buchi automata:
A_LIFT_ARRIVES = S0,
S1@ =(A_CALLED&!ALARM_ON -> S1),
S0 =(true -> S0 |A_CALLED&!ALARM_ON -> S1).
Composition:
A_LIFT_ARRIVES = A_CALLED || ALARM_ON || SYNC || A_LIFT_ARRIVES
State Space:
 2 * 2 * 2 * 3 = 2 ** 5
Composing...
-- States: 20 Transitions: 106 Memory used: 9146K
Composed in 18ms
After Tau elimination = 9 state
A_LIFT_ARRIVES minimising....
Minimised States: 6 in 1ms
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM || A_LIFT_ARRIVES
State Space:
 9 * 9 * 5 * 44 * 2 * 6 = 2 ** 21
LTL Property Check...
-- States: 641 Transitions: 1662 Memory used: 11243K
No LTL Property violations detected.
LTL Property Check in: 2ms
```

```
Formula !A_GETS_LIFT = (true U (A_WAITING & (false R (A_WAITING & !ALARM_ON))))
GBA 2 states 3 transitions
Buchi automata:
A_GETS_LIFT = S0,
S1@ =(A_WAITING&!ALARM_ON -> S1),
S0 =(true -> S0 |A_WAITING&!ALARM_ON -> S1).
Composition:
A_GETS_LIFT = A_WAITING || ALARM_ON || SYNC || A_GETS_LIFT
State Space:
 2 * 2 * 2 * 3 = 2 ** 5
Composing...
-- States: 20 Transitions: 88 Memory used: 9167K
Composed in 21ms
After Tau elimination = 9 state
A_GETS_LIFT minimising....
Minimised States: 6 in 0ms
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM || A_GETS_LIFT
State Space:
 9 * 9 * 5 * 44 * 2 * 6 = 2 ** 21
LTL Property Check...
-- States: 645 Transitions: 1688 Memory used: 11265K
No LTL Property violations detected.
LTL Property Check in: 2ms
```

```
Formula !MUTEX = (true U (A_IN_LIFT & B_IN_LIFT))
GBA 2 states 3 transitions
Buchi automata:
MUTEX = S0,
S1@ =(true -> S1),
S0 =(true -> S0 |A_IN_LIFT&B_IN_LIFT -> S1).
Composition:
MUTEX = A_IN_LIFT || B_IN_LIFT || SYNC || MUTEX
State Space:
 2 * 2 * 2 * 2 = 2 ** 4
Composing...
-- States: 16 Transitions: 105 Memory used: 9211K
Composed in 17ms
After Tau elimination = 8 state
MUTEX minimising...
Minimised States: 5 in 0ms
make DFA(MUTEX)
DFA(MUTEX) has 3 states.
MUTEX minimising...
Minimised States: 3 in 1ms
Composition:
LIFT_SYSTEM = USERS.a:USER || USERS.b:USER || QUEUE || LIFT || ALARM || MUTEX
State Space:
 9 * 9 * 5 * 44 * 2 * 3 = 2 ** 20
Analysing...
Depth 10 -- States: 292 Transitions: 636 Memory used: 8177K
No deadlocks/errors
Analysed in: 0ms
```

*nessuna violazione per quanto riguarda il check sulle LTL PROPERTIES*